

Trie; Radix Trie

Introducción al Trie:

Los Árboles Balanceados <imagen de goku aquí>

Previamente se había discutido el uso de árboles balanceados, debido a que garantizan la ejecución de operaciones de inserción, borrado y búsqueda en tiempo $O(\log n)$ en los peores casos. Y este es el caso en la mayoría de casos en que no contamos con ninguna otra información respecto los datos a manipular.

Pero ¿Y si tuviéramos información adicional?, por ejemplo, consideremos que nuestra estructura tratará con un tipo de dato en específico: STRING

Para ilustrar un caso de uso respecto nuestras estructuras de datos, se muestra como ejemplo el Auto-corrector.

Se busca que nuestro algoritmo logre determinar que una expresión escrita corresponda a palabras reales del idioma en cuestión, además, asumimos además que debe lograrse de forma eficiente respecto los recursos de memoria y computo utilizados. Adicionalmente, se desearía que un usuario pueda insertar sus propias expresiones, en caso que hayan algunas palabras, como nombres, que no hayan sido comprendidas extensivamente por nuestro algoritmo.

El diseño básico respecto al autocorrector suele ser intuitivo, haciendo uso de un diccionario como base, y el primer diseño, implica que cada nodo sea una palabra, sin embargo, esto se vuelve rápidamente muy ineficiente, pues significaría que cada nodo debería contener una String de longitud ilimitada*, con lo que el árbol tomaría una cantidad inmensa de almacenamiento, pues tomando una longitud de palabras promedio L , notaremos que el espacio de almacenamiento del árbol es proporcional al $n*L$, por lo que si notamos por m a la longitud máxima de alguna palabra, tenemos que el espacio en memoria utilizado por nuestra estructura será de la forma $O(n*m)$

Luego, para la búsqueda, notamos que una vez más, la longitud de cada palabra se vuelve relevante, pues para determinar si la palabra buscada es la actual, hay que comparar individualmente cada carácter, lo que de inmediato nos revela que habrá una gran cantidad de comparaciones repetidas entre palabras.

Por ejemplo, las palabras **almanaque** y **almacen**, requieren llegar hasta la quinta comparación para determinar la diferencia.

Entonces, se hace evidente que tenemos que buscar otra estrategia.

¡A COMPRIMIR!

Como se habrá notado, las palabras que consideramos similares, lo son porque tienen la misma iniciación de letras, por lo que si pudiéramos hacer que de los prefijos desprendieran las palabras de algún modo, se alcanzaría una optimización relevante...

Trie, buscando en Strings desde 1959

Pronunciado “try” por su similitud a “tree” y su uso designado a operaciones “retrieval”, se diseñaron como una forma compacta y eficiente de realizar búsquedas sobre Strings, reduciendo las redundancias al almacenar prefijos comunes una única vez.

Aunque estábamos acostumbrados a los árboles binarios, este es un caso en que no son una opción considerable; en su lugar, se requiere de árboles n-arios, de modo que son las aristas las que son marcadas con una letra del abecedario, y los nodos solo actúan como puntos intermedios de conexión entre letras, o como indicadores de clave alcanzada (indican si al alcanzar dicho nodo, se ha encontrado un valor almacenado en ese punto de la estructura).

Así, si nuestro alfabeto Sigma tiene k símbolos, nuestro Trie será un árbol k -ario, donde los nodos no tendrán más información más que una marca que indique si hay una palabra registrada con término en éste, o no.

Luego, a partir de lo anterior, es que notamos que, aunque es cierto que todas las hojas del Trie serán nodos marcados, también habrán otros tantos nodos internos que estén marcados, por lo que nuestros nodos tendrán una cualidad más allá de ser hojas o nodos internos, a estos les llamaremos *nodos clave*.

Además, cada letra descendiente del mismo nodo a lo largo del camino, tendrá un mismo prefijo, siendo el nodo raíz especial, en el sentido que corresponde a la cadena vacía.

Y aunque esto es una introducción para lo que son los Tries, es de notar que también son utilizados para almacenar e indexar palabras a lo largo de textos, por lo que necesitaremos introducir ya sea un contador, o un listado de posiciones, a los nodos clave marcados.

¿Por qué es esto mejor?

Comencemos con un análisis de memoria utilizada:

Comparemos entonces, el espacio que utilizaría la estructura implementada como Árbol Binario de Búsqueda:

- Considerando caracteres codificados uniformemente, cada uno de ellos usaría una cantidad fija de bits, y en el caso de los BST, sería requerido un símbolo adicional para indicar una finalización de palabra.
- Los tries únicamente tienen enlaces para nodos existentes, considerando una cantidad fija de posibles apuntadores nulos,
- Dentro de un trie, cada nodo tendrá k enlaces, con k el tamaño del alfabeto usado.
- Cada enlace tomará, el espacio de referencia para BST, mientras que en Trie tomará además espacio adicional para identificar el nodo del que proviene.
- Cada nodo en BST requiere de un espacio indicado para cada uno de sus caracteres, mientras que en Trie solo se requiere el espacio del indicador de si es nodo intermedio/final o no

De esto, con ejemplares relativamente pequeños en cuenta de palabras (por ejemplo 10), la implementación con BST resulta mejor respecto uso de memoria, sin embargo al elevar los datos por almacenar, es que Trie rápidamente retoman la delantera.

De esto, se hace la notación que, según la tasa de nodos clave/totales se eleva, es que conocemos que un trie será en términos prácticos más eficiente, debido que gracias al funcionamiento de los caminos en Trie, significará que habrá más de una palabra almacenada en el mismo camino, mientras que para BST se debe construir y comparar cada palabra individualmente. A esto se le conoce como compresión, misma estrategia que se había mencionado al inicio de esta sección.

Es cuando no podemos aprovecharnos de la compresión, que nos aproximamos al peor caso para Tries, y éste se da cuando contamos con una ausencia de nodos clave, y la basta mayoría

de nodos marcados son los finales, u hojas. Sin embargo, aunque esto puede llegar a suceder, al llevarlo a la aplicación, es muy poco posible que ocurra, en cuyo caso, aún son comparables al uso de memoria indicado por un BST.

BST $O(n)$ vs $O(n/d * k)$ Trie

Buscando...

Suponiendo que ya se ha construido un Trie... ¿Cómo se busca dentro de éste, y cuánto toma?

La estrategia es sencilla, sin embargo, implica recorrer el árbol generado, un carácter a la vez respecto la clave buscada.

Comenzando por el nodo raíz 'r' del árbol Trie 'T', avanzaremos por los caminos disponibles según lo indique el i-ésimo carácter de la palabra buscada, hasta que hayamos acabado con los caracteres de la palabra, o se alcance un nodo sin un enlace correspondiente al carácter indicado.

Este es el procedimiento más importante del Trie, pues el resto serán implementados a partir de éste.

¿Cómo es el rendimiento de la búsqueda implementada sobre Tries?

La complejidad de realizar éste proceso, es $O(d)$, con d la máxima cantidad de caracteres presentes en una palabra registrada.

- Consideramos que comparar 2 caracteres cualesquiera toma $O(1)$
- La búsqueda del siguiente carácter tiene un tiempo distinto dependiendo de la implementación (recordemos k como la cantidad de caracteres comprendidos por el alfabeto)
 - Suponiendo el uso de una estructura *HashTable* para las aristas, obtenemos una complejidad amortizada $O(1)$
 - Suponiendo el uso de un BST, se tiene una complejidad de $O(\log k)$
 - Suponiendo el uso de un arreglo, se tiene complejidad $O(k)$
- El manejo de Strings puede provocar ineficiencias
 - String suele ser implementado como inmutable, por lo que requerir la extracción de sub-strings puede ocasionar que cada paso de la búsqueda tome $O(d)$ en memoria y tiempo
 - Utilizar la indexación de caracteres en String hace que solo tome $O(1)$

Inserción – $O(d)$

Como se mencionó anteriormente, la inserción en Tries sigue una doctrina muy similar al procedimiento de búsqueda, y tiene escenarios a considerar:

- El camino correspondiente a la palabra ya existe (se ha de marcar el nodo final)
- El camino corresponde solo a una sub-string de la palabra (se han de agregar nuevos nodos)

El procedimiento es de la siguiente forma:

1. Recorrer el Trie existente
2. Si hemos alcanzado el final del string buscado, el último nodo encontrado será marcado

3. Si hemos alcanzado el final del camino con caracteres pendientes, agregaremos cada nodo individualmente, esta operación es $O(1)$ por cada nodo, asumiendo que se utilizan HashTable para las aristas
 - a. Una vez se acaban las letras de la palabra, el último nodo ha de ser marcado

Remover – $O(d)$

El remover tiene consideraciones adicionales, pues podemos usar un procedimiento veloz que puede desperdiciar memoria, o uno que es menos eficiente en tiempo, pero que evita cualquier uso innecesario de memoria

El primero consiste en simplemente desmarcar el nodo clave correspondiente a la palabra, lo que provoca que el camino sobrante persista sin ser utilizado, lo cual se agrava si son nodos hoja, puesto que estos maximizan la cantidad de nodos desperdiciados.

El segundo implementa adicionalmente un procedimiento de poda, de modo que todo nodo al final del camino sea eliminado hasta alcanzar el nodo clave previo

Prefijos largos – $O(d)$

Suele darse la ocasión, que aún si no se encuentra la palabra buscada, aún queremos saber la porción más grande que sí está contenida, esto puede lograrse aprovechando el procedimiento de búsqueda, siguiendo el recorrido, y una vez se ha alcanzado el final de este, determinamos:

- Si el nodo final es nodo clave, devolvemos la string correspondiente al camino recorrido
- Si el nodo final no es nodo clave, retrocedemos en el camino hasta encontrar uno que sí lo sea, y posteriormente devolvemos la string correspondiente al camino desde la raíz hasta el nodo clave

Claves con Prefijo

De forma simétrica al procedimiento previo, sabemos que hay palabras que comparten prefijos, por lo que sería conveniente realizar búsquedas de palabras que correspondan a un mismo prefijo.

Y si lo consideramos, resulta hasta intuitivo que esta es una posibilidad gracias a la forma en que se estructuran los Tries, tal que algunos le nombran “árboles de prefijos”.

Ya vimos que Trie logra comprimir Strings al compartir los prefijos comunes, por lo que puede implementarse del siguiente modo:

1. Se realiza el recorrido del prefijo indicado en el Trie
2. Una vez alcanzado el nodo final del prefijo, se realiza un recorrido de árbol, tomando como raíz el final del prefijo, y a lo largo del recorrido de árbol, se agregará cada palabra encontrada (identificada gracias a los nodos clave)

Este es un procedimiento más complicado que el resto, por lo que también tiene una complejidad de cómputo superior.

- No sabemos exactamente cuántas claves tiene como descendientes un prefijo, por lo que resta estimar. Así, si sabemos que nuestro Trie tiene n claves, con una longitud máxima d , entonces la búsqueda de todas las claves a partir de un prefijo no-vacío de longitud “ m ” sería $O(m * n^{(d-m)})$, que corresponde a un árbol degenerado Trie donde todas las palabras tienen el mismo prefijo

- Si usáramos como prefijo a la cadena vacía, entonces este procedimiento tendría que encontrar todas las claves registradas en el Trie, lo que tomaría tiempo $O(n * d)$, que sería el peor caso absoluto para esta operación

Tries ¿Cuándo sí? ¿Cuándo no?

Aunque hasta ahora solo hemos visto cosas buenas de los Tries, ninguna estructura es perfecta, y el Trie no es la excepción

- Un trie puede ser menos eficiente en la búsqueda que un HashTable, pues mientras el Trie requiere $O(d)$ para recorrer el camino, HashTable hace las consultas en tiempo amortizado $O(1)$
- Mientras que HashTable suele permanecer en una sección continua de memoria, Trie por su naturaleza de nodos puede estar repartida a lo largo y ancho del Heap, por lo que no sería aprovechada la cualidad de localidad (Es más eficiente buscar en un bloque de memoria próximo que en uno alejado)
- Aunque cualquier* valor puede ser convertido a String, resulta notable que por ejemplo, al tratar con valores de punto flotante, es posible generar caminos largos e inútiles, como serían los números con repetición periódica, números trascendentales, o resultados aproximados de operaciones con margen de error, propios de la aritmética computarizada ($0.1+0.2 = 0.30000000000000004$)
- Dependiendo de la implementación de Trie, puede encontrarse un desperdicio de memoria por la necesidad de cada nodo de referenciar k nodos correspondientes a los símbolos del alfabeto, aún si se tratase de un nodo hoja (que no tiene hijos)

Es por esto, y lo analizado previamente, que los Tries tienen un rendimiento óptimo al tratar con búsquedas frecuentes que explotan las similitudes por prefijos repetidos, mientras que se recomendaría el uso de HashTable para realizar consultas sobre datos almacenados en dispositivos con bajo rendimiento de consulta, o si es necesario explotar la cualidad de localidad.

Entonces, String y las operaciones que se pueden realizar sobre esta son el campo de dominio de los Trie, aunque la compresión debe ser considerada, pues si no se puede aprovechar, nuestro rendimiento oscilaría desde $O(n * d)$ hasta $O(n * d * k)$, con d la longitud promedio de las palabras, k la cantidad de símbolos en el alfabeto, y n la cantidad de claves almacenadas en el Trie.

También, conocemos del desperdicio causado por los nodos nulos, y algunas soluciones, como el uso de diccionarios para referenciar solo aquellos nodos que no sean nulos, sin embargo esto tiene sus propios costos, como reducir la velocidad de acceso al siguiente nodo, o de eliminar registros (como sería el caso en la poda de nodos)

MEJORAS GENTE, MEJORAS

Algunas alternativas se formaron para intentar lidiar con esto, como los TST (Ternary Search Tree), que sacrifican rendimiento de cómputo por un ahorro de memoria, o *RADIX TRIE*

Usualmente en Trie, la mayoría de los nodos no son clave, sino que actúan como camino puente entre nodos que sí son claves, y bastantes de esos puentes son necesarios, sin embargo no lo son todos, y notar esto es relevante para cadenas de textos alargadas donde la separación entre nodos clave es bastante considerable, y realmente los nodos intermedios son

simples puentes (nodos que tienen un único hijo). Así, identificamos que hay nodos intermedios puente, y nodos intermedio que ramifican.

Los Trie Radix (Patricia) reconocen esta falencia, y la solucionan con la idea que los caminos pueden comprimirse, tal que una serie consecutiva de nodos intermedios de hijo unico que unan en uno solo, que será llamado *nodo de paso*.

¿Cuánto cambia las cosas esto?

Las operaciones que se habían definido originalmente para Trie siguen funcionando con la misma idea, sin embargo deben pasar por modificaciones para operar adecuadamente, puesto que al encontrarse con nodos de paso, no podrá acceder a cada nodo individualmente el recorrido, sino que ha de pasar por multiples caracteres a la vez en busqueda de coincidencias. Además, se conserva la propiedad de prefijos, por lo que conocemos que si el i-esimo caracter de la arista está presente, no hay otra palabra que comparta ese mismo prefijo y no sea comprendido por el camino recorrido.

Para buscar respecto los nodos de paso entonces, podemos buscar respecto cada caracter en el String, cosa que podemos hacer usando Busqueda Binaria, misma que tomará $O(\log k)$, siendo k la cantidad de simbolos en el alfabeto, y ya que tenemos una cantidad fija y finita de símbolos, será un resultado constante, por lo que realizar la busqueda tomaría $O(1)$ en tiempo de computo amortizado, y no se requeriría memoria adicional.

Podrían usarse arreglos ordenados para almacenar los valores de los nodos de paso, pero esto implicaría a futuro que las inserciones serían lineales, lo cual no suele ser optimo para hacer una larga serie de inserciones.

Otra implemetación podría ser con HashTable, como antes, almacenando las aristas directamente en la estructura, lo que permitiría una consulta en tiempo amortizado $O(1)$

Independientemente de esto, el primer paso es comparar los caracteres del string buscado con los del amacenado en el nodo, lo que tiene 4 posibilidades:

1. Coincidencia perfecta (Todos los caracteres coinciden)
2. Los primeros caracteres coinciden, pero no el resto
 - a. La palabra buscada queda rebasada por la almacenada
 - b. Divergencia de caracter tras varias coincidencias
3. Divercencia total (no comparten ni el primer caracter)

BUSQUEDA – $O(d)$

Es de forma similar a la impementación para Trie basico, con la diferencia que además debemos poder recorrer las aristas

INSERCION – $O(d)$

Los casos mas complicados de manejar son los del punto 2, pues requieren subdividir el nodo de paso con la string previamente comprimida , sin embargo continúa con una logica similar a la de Trie basico

REMOVED – $O(d)$

Esta operación sigue la misma logica que en Trie basico, sin embargo, tiene la propiedad que al terminar de realizar una remoción, tiene el potencial de realizar una compresión, pues si

elimina un nodo clave que a su vez tiene otro nodo clave debajo, puede comprimir en sí los nodos debajo suyo hasta alcanzar el clave.

En el caso de la poda de nodos, (cuando se remueve un nodo clave que también es hoja), notamos que se ve optimizada, puesto que solo debe eliminar un nodo adicional, en vez de realizar una serie de eliminaciones consecutivas.

PREFIJO COMUN LARGO

Se vuelve cuestion de adaptar el procedimiento para Trie original, puesto que se sigue la misma lógica, adaptada para realizar búsquedas sobre aristas con mas de un caracter.

CLAVES DE PREFIJO COMUN

En el caso de Radix Trie se complica un tanto, pues es posible que el prefijo indicado se encuentre comprendido parcialmente por aristas, es decir, que el prefijo se encuentre “separado” a lo largo de varias aristas.

Por lo que el procedimiento debería seguir, caracter a caracter, el camino indicado por el prefijo, y una vez alcanzado el final asumir al nodo actual como raíz de la búsqueda en arbol mediante recorrido.

APLICACIONES

- Autocorrectores
 - Originalmente se implementaron con *Bloom Filters*, sin embargo posteriormente fueron re-implementados con alternativas mas eficientes, como Tries.
 - Primero, se inserta toda palabra del idioma al Trie
 - Posteriormente, por cada palabra escrita, se busca en el Trie del idioma, y si no la encuentra como clave valida, lo indica
 - Si además queremos proponer una corrección,
- Autcompletado
- Ordenado de Strings